gayathri@Gayathri: ~/mini_sh

```
gayathri@Gayathri:~$ mkdir mini_shell
cd mini_shell
gayathri@Gayathri:~/mini_shell$ gedit shell.c

** (gedit:465): WARNING **: 09:46:41.450: Could not load Peas repository: Typelib file for namespace 'Peas', version '1.0' not found

** (gedit:465): WARNING **: 09:46:41.450: Could not load PeasGtk repository: Typelib file for namespace 'PeasGtk', version '1.0' not
found
gayathri@Gayathri:~/mini_shell$ nano shell.c
gayathri@Gayathri:~/mini_shell$ ls
shell.c
gayathri@Gayathri:~/mini_shell$ cat shell.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <termios.h>
#include <sys/wait.h>

#define INITIAL_BUFFER_SIZE 64

/* History linked-list node */
struct Node
{
    char *command;
    struct Node *next;
};

/* Head and tail of history */
struct Node *history_head = NULL;
struct Node *history_tail = NULL;

/* Original terminal settings */
struct termios original_terminal;
```

gayathri@Gayathri: ~/mini_sh

```
struct Node *history_tail = NULL;

/* Original terminal settings */
struct termios original_terminal;

/* Enable raw mode */
void enable_raw_mode()
{
    struct termios raw;

    tcgetattr(STDIN_FILENO, &original_terminal);

    raw = original_terminal;

    raw.c_lflag &= ~(ICANON | ECHO);

    tcsetattr(STDIN_FILENO, TCSAFLUSH, &raw);
}

/* Restore normal terminal mode */
void disable_raw_mode()
{
    tcsetattr(STDIN_FILENO, TCSAFLUSH, &original_terminal);
}

/* Add command to history */
void add_history(const char *command)
{
    if (command == NULL || strlen(command) == 0)
        return;

    struct Node *new_node = malloc(sizeof(struct Node));

    if (new_node == NULL)
    {
```

gayathri@Gayathri: ~/mini_sh

```
        return;
    }

    new_node->command = strdup(command);

    if (new_node->command == NULL)
    {
        perror("strdup");
        free(new_node);
        return;
    }

    new_node->next = NULL;

    if (history_head == NULL)
    {
        history_head = new_node;
        history_tail = new_node;
    }
    else
    {
        history_tail->next = new_node;
        history_tail = new_node;
    }
}

/* Count history entries */
int history_count()
{
    int count = 0;
    struct Node *current = history_head;

    while (current != NULL)
    {
        count++;
```

gayathri@Gayathri: ~/mini_sh

```
    {
        count++;
        current = current->next;
    }

    return count;
}

/* Get command at a particular history position */
char *get_history_command(int position)
{
    struct Node *current = history_head;
    int index = 0;

    while (current != NULL)
    {
        if (index == position)
            return current->command;

        index++;
        current = current->next;
    }

    return NULL;
}

/* Free entire history linked list */
void free_history()
{
    struct Node *current = history_head;

    while (current != NULL)
    {
        struct Node *next = current->next;
```

gayathri@Gayathri: ~/mini_sh

```
        struct Node *next = current->next;

        free(current->command);
        free(current);

        current = next;
    }

    history_head = NULL;
    history_tail = NULL;
}

/* Clear current line */
void clear_current_line()
{
    printf("\r\033[K$ ");
    fflush(stdout);
}

/* Read a command with arrow-key history */
char *read_command()
{
    size_t buffer_size = INITIAL_BUFFER_SIZE;
    size_t length = 0;

    char *buffer = malloc(buffer_size);

    if (buffer == NULL)
    {
        perror("malloc");
        exit(EXIT_FAILURE);
    }

    buffer[0] = '\0';
```

gayathri@Gayathri: ~/mini_sh

```
    int history_position = total_history;

    printf("$ ");
    fflush(stdout);

    while (1)
    {
        char c;

        if (read(STDIN_FILENO, &c, 1) != 1)
            continue;

        /* ENTER */
        if (c == '\n' || c == '\r')
        {
            printf("\n");
            buffer[length] = '\0';
            return buffer;
        }

        /* BACKSPACE */
        if (c == 127 || c == 8)
        {
            if (length > 0)
            {
                length--;
                buffer[length] = '\0';

                printf("\b \b");
                fflush(stdout);
            }

            continue;
        }
```

gayathri@Gayathri: ~/mini_sh

```
        }

        /* ESCAPE SEQUENCE */
        if (c == '\033')
        {
            char sequence[2];

            if (read(STDIN_FILENO, &sequence[0], 1) != 1)
                continue;

            if (read(STDIN_FILENO, &sequence[1], 1) != 1)
                continue;

            /* UP ARROW: ESC [ A */
            if (sequence[0] == '[' && sequence[1] == 'A')
            {
                if (history_position > 0)
                {
                    history_position--;

                    char *old_command =
                        get_history_command(history_position);

                    if (old_command != NULL)
                    {
                        length = strlen(old_command);

                        while (length + 1 >= buffer_size)
                        {
                            buffer_size *= 2;

                            char *temp =
                                realloc(buffer, buffer_size);
```

gayathri@Gayathri: ~/mini_sh

```
                                free(buffer);
                                perror("realloc");
                                exit(EXIT_FAILURE);
                            }

                            buffer = temp;
                        }

                        strcpy(buffer, old_command);

                        clear_current_line();

                        printf("%s", buffer);
                        fflush(stdout);
                    }
                }

                continue;
            }

            /* DOWN ARROW: ESC [ B */
            if (sequence[0] == '[' && sequence[1] == 'B')
            {
                if (history_position < total_history - 1)
                {
                    history_position++;

                    char *next_command =
                        get_history_command(history_position);

                    if (next_command != NULL)
                    {
                        length = strlen(next_command);
```

gayathri@Gayathri: ~/mini_sh

```
                            free(buffer);
                            perror("realloc");
                            exit(EXIT_FAILURE);
                        }

                        buffer = temp;
                    }

                    strcpy(buffer, old_command);

                    clear_current_line();

                    printf("%s", buffer);
                    fflush(stdout);
                }
            }

            continue;
        }

        /* DOWN ARROW: ESC [ B */
        if (sequence[0] == '[' && sequence[1] == 'B')
        {
            if (history_position < total_history - 1)
            {
                history_position++;

                char *next_command =
                    get_history_command(history_position);

                if (next_command != NULL)
                {
                    length = strlen(next_command);

                    while (length + 1 >= buffer_size)
```

gayathri@Gayathri: ~/mini_sh

```
                        buffer_size *= 2;

                        char *temp =
                            realloc(buffer, buffer_size);

                        if (temp == NULL)
                        {
                            free(buffer);
                            perror("realloc");
                            exit(EXIT_FAILURE);
                        }

                        buffer = temp;
                    }

                    strcpy(buffer, next_command);

                    clear_current_line();

                    printf("%s", buffer);
                    fflush(stdout);
                }
            }
            else
            {
                history_position = total_history;
                length = 0;
                buffer[0] = '\0';

                clear_current_line();
                fflush(stdout);
            }

            continue;
        }
```

gayathri@Gayathri: ~/mini_sh

```
        /* Normal character */
        if (c >= 32 && c <= 126)
        {
            /* Resize buffer when necessary */
            if (length + 1 >= buffer_size)
            {
                buffer_size *= 2;

                char *temp =
                    realloc(buffer, buffer_size);

                if (temp == NULL)
                {
                    free(buffer);
                    perror("realloc");
                    exit(EXIT_FAILURE);
                }

                buffer = temp;
            }

            buffer[length] = c;
            length++;

            buffer[length] = '\0';

            putchar(c);
            fflush(stdout);
        }
    }
}

/* Execute command */
void execute_command(char *command)
{
```

gayathri@Gayathri: ~/mini_sh

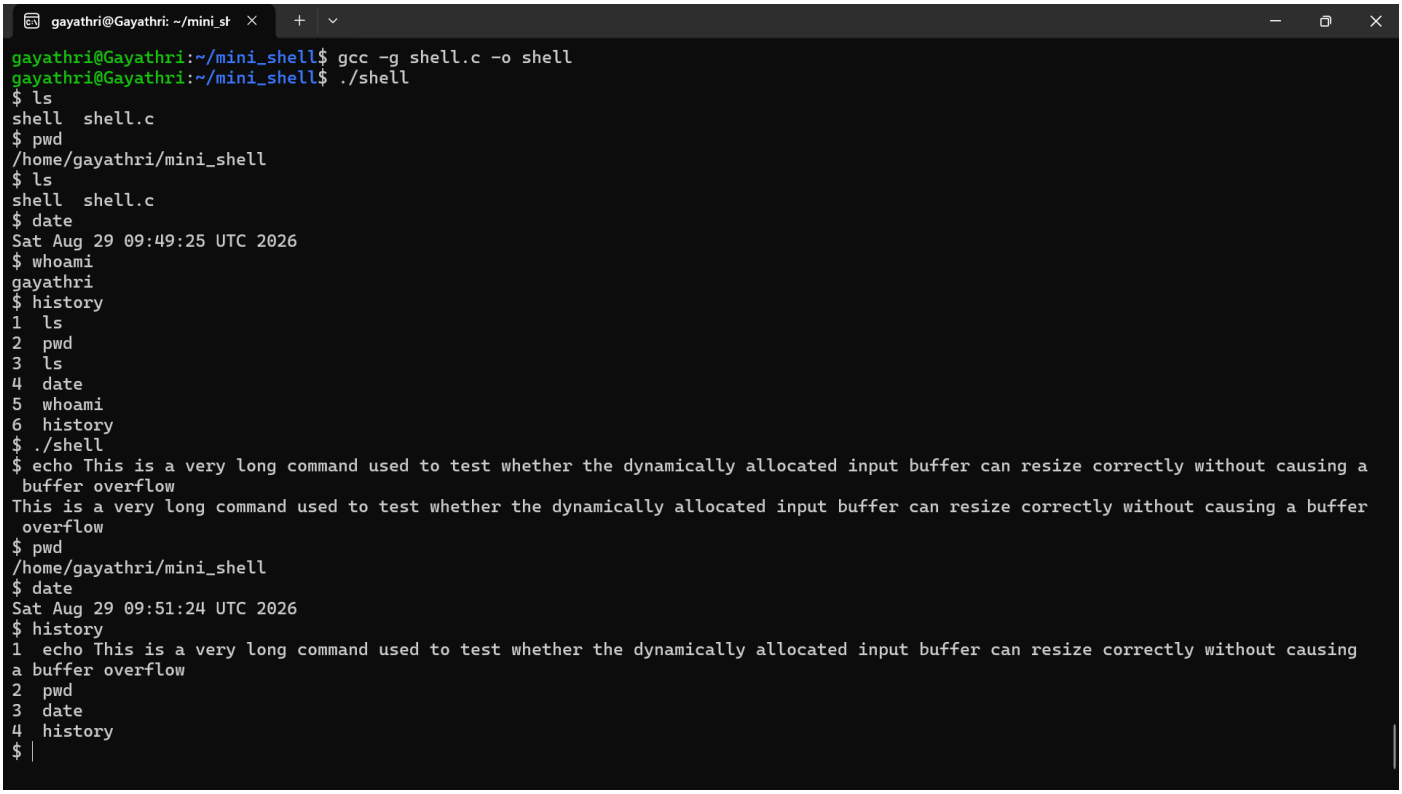
```
        return;

    /* Exit command */
    if (strcmp(command, "exit") == 0)
    {
        disable_raw_mode();
        free_history();
        exit(0);
    }

    /* History command */
    if (strcmp(command, "history") == 0)
    {
        struct Node *current = history_head;
        int number = 1;

        while (current != NULL)
        {
            printf("%d  %s\n", number, current->command);

            number++;
            current = current->next;
        }

        return;
    }

    /* Create a copy because strtok modifies the string */
    char *command_copy = strdup(command);

    if (command_copy == NULL)
    {
        perror("strdup");
        return;
    }
```

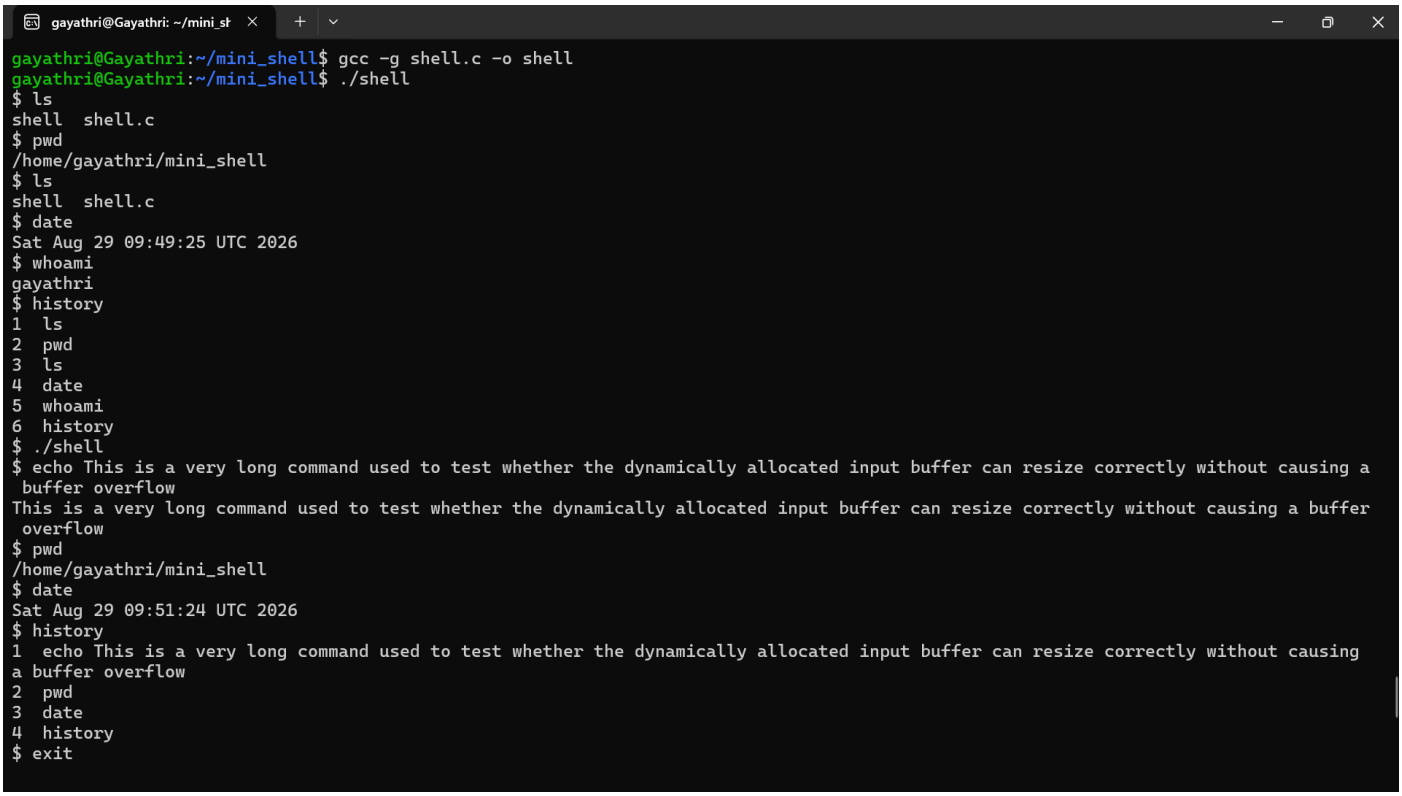
gayathri@Gayathri: ~/mini_sh

```
    char *args[100];
    int argument_count = 0;

    char *token = strtok(command_copy, " ");

    while (token != NULL && argument_count < 99)
    {
        args[argument_count] = token;
        argument_count++;

        token = strtok(NULL, " ");
    }

    args[argument_count] = NULL;

    if (argument_count == 0)
    {
        free(command_copy);
        return;
    }

    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
    }
    else if (pid == 0)
    {
        execvp(args[0], args);

        perror("execvp");
        exit(EXIT_FAILURE);
    }
    else
```

gayathri@Gayathri: ~/mini_sh

```
    free(command_copy);
}

int main()
{
    enable_raw_mode();

    while (1)
    {
        char *command = read_command();

        if (strlen(command) > 0)
        {
            add_history(command);
            execute_command(command);
        }

        free(command);
    }

    disable_raw_mode();
    free_history();

    return 0;
}
gayathri@Gayathri:~/mini_shell$ gcc -g shell.c -o shell
gayathri@Gayathri:~/mini_shell$ ./shell
$ ls
shell  shell.c
$ pwd
/home/gayathri/mini_shell
$ ls
shell  shell.c
$ date
Sat Aug 29 09:49:25 UTC 2026
```


gayathri@Gayathri: ~/mini_sh
gayathri@Gayathri:~/mini_shell$ gcc -g shell.c -o shell
gayathri@Gayathri:~/mini_shell$ ./shell
$ ls
shell shell.c
$ pwd
/home/gayathri/mini_shell
$ ls
shell shell.c
$ date
Sat Aug 29 09:49:25 UTC 2026
$ whoami
gayathri
$ history
1 ls
2 pwd
3 ls
4 date
5 whoami
6 history
$ ./shell
$ echo This is a very long command used to test whether the dynamically allocated input buffer can resize correctly without causing a
 buffer overflow
This is a very long command used to test whether the dynamically allocated input buffer can resize correctly without causing a buffer
 overflow
$ pwd
/home/gayathri/mini_shell
$ date
Sat Aug 29 09:51:24 UTC 2026
$ history
1 echo This is a very long command used to test whether the dynamically allocated input buffer can resize correctly without causing
a buffer overflow
2 pwd
3 date
4 history
$


gayathri@Gayathri: ~/mini_sh
gayathri@Gayathri:~/mini_shell$ gcc -g shell.c -o shell
gayathri@Gayathri:~/mini_shell$ ./shell
$ ls
shell shell.c
$ pwd
/home/gayathri/mini_shell
$ ls
shell shell.c
$ date
Sat Aug 29 09:49:25 UTC 2026
$ whoami
gayathri
$ history
1 ls
2 pwd
3 ls
4 date
5 whoami
6 history
$ ./shell
$ echo This is a very long command used to test whether the dynamically allocated input buffer can resize correctly without causing a
 buffer overflow
This is a very long command used to test whether the dynamically allocated input buffer can resize correctly without causing a buffer
 overflow
$ pwd
/home/gayathri/mini_shell
$ date
Sat Aug 29 09:51:24 UTC 2026
$ history
1 echo This is a very long command used to test whether the dynamically allocated input buffer can resize correctly without causing
a buffer overflow
2 pwd
3 date
4 history
$ exit